ANTES DE COMENZAR EL EXAMEN

Modalidad del examen: en PC, Centro de Cómputos, sin acceso a Internet

Tiempo asignado: 120 minutos

Pautas:

- Tamaño de letra: ARIAL 11
- Responder debajo de la consigna.
- No sumará puntaje respuestas que no respeten lo que pide el ejercicio
- Formato de entrega: documento PDF
-

Nombre del archivo: APELLIDO_PROMOCION

Examen FINAL
Análisis y Diseño de Sistemas II
Diciembre 2025

Nombre y Apellido: FEDERICO RODRIGUEZ

Parte	Tema	Aspectos a evaluar
I	**Patrones de diseño**	Capacidad para identificar patrones de diseño en distintas situaciones e indicar su propósito

Ejercicio 1
Situación:Un sistema de e-commerce permite recorrer productos de distintas formas:

- En orden de precio
- En orden alfabético
- Solo productos disponibles en stock

El cliente del sistema quiere poder cambiar la forma de recorrido sin modificar el código que consume la colección.

Pregunta:
1.1 ¿Qué patrón de diseño permite resolver este requerimiento y por qué? (4 líneas)
El patron que resuelve este requerimiento es el Iterator ya que te permite recorrer ola colección de objetos sin exponer los detalles de la estructura de cada objeto. ✓

1.2 Describir, con un modelo dinámico, como interactúan los elementos principales del patrón
Sin respuesta

Ejercicio 2

Situación: Se está desarrollando un sistema con arquitectura REST en Spring Boot para la gestión de usuarios.
El sistema expone los siguientes endpoints:

- `POST /users` → crear usuario
- `PUT /users/{id}` → actualizar usuario
- `DELETE /users/{id}` → eliminar usuario
- `PATCH /users/{id}/activate` → activar/desactivar usuario

Actualmente, el controlador REST contiene lógica del tipo:

- Validación del request
- Decisión de qué acción ejecutar
- Llamadas directas a distintos métodos del servicio
- Registro de auditoría (log de la acción realizada)

Se detecta que:

- El controlador tiene demasiada lógica.
- Cada vez que se agrega una nueva operación, el controlador debe modificarse.
- Se quiere incorporar en el futuro:
    - Auditoría uniforme
    - Posibilidad de reintentar operaciones
    - Ejecución diferida de algunas acciones

Pregunta:

2.1 Proponer una solución basada en el patrón Command, indicando:

- Qué rol cumple:
    - el Controller: capta la request y instancia la solicitud en un objeto Command.
    - la interfaz Command: se definen todos los command posibles que puede haber en el sistema.
    - los ConcreteCommand: se definen las llamadas al Receiver dependiendo del tipo de command que es pasando los parametro correctos. ✓

- el Receiver: se ejecuta la logica de la solicitud realizada.

2.2 ¿Cómo se modelaría una operación del CRUD como un Command? (por ejemplo, *CreateUserCommand*). No es necesario escribir código, solo describir clases y responsabilidades.

La clase Controller toma la solicitud y crea un objeto para que puede despues manipularse, esta solo realiza un “command.execute()”.
La interfaz command se van a encontrar todos los tipos de Commands que pueden haber en el sistema, sin definir la lógica que de su implementación.
Las clases ConcreteCommand: estas implementan la interfaz Command y definen la lógica de que parámetros tienen que pasarle al Receiver.
Receiver: este realiza la acción que le llega en el objeto command,en este caso actualiza, elimina o crea usuarios. ✓

Parte	Tema	Aspectos a evaluar
II	**Documentación de la arquitectura**	Conocimiento sobre las vistas para modelar una arquitectura

Ejercicio 3

3.1 ¿Por qué es importante documentar la arquitectura usando múltiples vistas? (4 líneas)

Las vistas permiten comunicar distintas perspectivas del sistema a distintos interesados

Es importante documentar la arquitectura ya que es fundamental dejar pactado la estructura de que va a tener el proyecto, los pasos que hay seguir para desarrollarlo y las restricciones impuestas. Con el fin de tener una arquitectura solida y que un grupo de desarrolladores pueda codificar de forma paralelo múltiples componentes y si quiere modificar/agregar algún componente la estructura del proyecto no se vea afectada.

proyecto??

qué es una arquitectura sólida?

3.2 Considerando el modelo 4+1 de vistas de arquitectura:

3.2.1 Indicar en qué vistas se representan principalmente los requerimientos funcionales y justificar.

La **vista lógica y la vista de casos de usos**, lo que te permite centrarte en que es lo que va a ser el sistema y los componentes además de cómo van a interactuar entre si y como va a responder a diferentes escenarios basándose en los casos de uso. dividiendo a los componentes en funciones específicas según so logica.

3.2.2 Indicar en qué vistas se analizan los requerimientos no funcionales como rendimiento, escalabilidad y disponibilidad.

La vista de implementación que es la que se centra en la relación de los componentes del sistema con la infraestructura (servidores, computadores, redes)., o sea en donde va a correr el sistema

vista de procesos, de despliegue

Parte	Tema	Aspectos a evaluar
III	**Proyecto**	Capacidad de relacionar la teoría de procesos de desarrollo con la implementación práctica de un proyecto a pequeña escala

Ejercicio 4

Describa en 8-10 líneas, cómo se consideraron las características del Proceso Unificado de Desarrollo de Software en el proyecto final

En el proyecto se pactaron, 2 iteraciones con el fin de minimizar los riegos de fallar en la finalización ya que se pueden detectar los errores a tiempo y responder a los requerimientos cambiantes que fueron surgiendo en el ciclo de vida del proyecto (o sea el cuatrimestre), incrementado en cada iteración las funcionalidades de nuestro proyecto y agregando mas complejidad al mismo (añadiendo patrones de diseño y nuevas funcionalidades).
En el caso de la arquitectura esta tuvo una estrecha relación con los casos de usos ya que se tuvo que hacer un análisis y diseño de manera tal que pudiera soportar todos los casos de usos seleccionados y su vez que permitiera añadir nuevos patrones de diseño y combinarlo con otra arquitectura, elegimos la Arquitectura MVC y la combinamos con la arquitectura reflexiva, añadiendo patrones de diseño como el Facade y el Singleton entre otros.
Y para detectar las funcionalidades de nuestro sistema hicimos un análisis de los casos de usos como anteriormente mencionados con la búsqueda de seleccionar los mas atractivos y que sean acordes a las tecnologias que ibamos a usar.

se consideró un c.u y se lo analizó, diseñó, implementó y probó
eso es "estar basado en c.u"

Ejercicio 5

El Proceso Unificado (RUP) organiza el desarrollo de software en fases y workflows (disciplinas).

5.1 Explicar qué se entiende por workflow dentro del RUP. (2 líneas)

qué es una buena labor?

Se entiende por la dinámica que hay que seguir para desarrollar una buena labor, identificando requerimiento, analizando las vistas arquitectónicas y el código para realizar un SW mínimo, completo y extensible. No.

5.2 Indicar cuáles son los workflows principales del Proceso Unificado.

- Modelado de negocio

- Análisis de requerimiento con ayuda de los Casos de Usos
- Análisis y diseño
- Implementación
- Pruebas
- Despliegue
- Gestión de configuraciones
- Gestión de proyecto
- Entorno

Ejercicio 6

6.1 Describa los 2 modelos construidos en el proyecto de la arquitectura de software (6 líneas).

se pide modelos arquitectónicos, no de diseño

Se utilizo por un lado el Modelo estático para identificar las clases y atributos y como interactúan con otras clases mediante asociaciones o dependencias, mas precisamente utilizando el diagrama de clases.
Por otro lado, utilizamos el Modelo dinámico para entender el comportamiento de los componentes y como interactúan con otros en tiempo real, poniendo especial énfasis en la secuencia y colaboración de los componentes, utilizando diagrama de secuencia y Colaboración.

Parte	Tema	Aspectos a evaluar
IV	Patrones Arquitectónicos	Capacidad de identificar patrones arquitectónicos para aplicar en situaciones y separación de responsabilidades.

Ejercicio 7

7.1 Explicar en qué consiste una Arquitectura Dirigida por Eventos y cuál es su idea central. (6 líneas)

Básicamente la EDA es una arquitectura de tipo distribuida que tiene por objetivo la comunicación entre componentes independientes a través de eventos, que estos eventos pueden ser cambio de estados o acciones, asíncronos la mayoría. ✓
Permitiendo agregar o modificar componentes sin la necesidad afectar el sistema

7.2 Describir e ilustrar sus componentes

Los principales componentes son: ✓

- Productores: detectan un cambio de estado o accion y producen un eventos que es mandado a los consumidores.
- Consumidores: reciben un evento y producen acciones que estas son independientes de las de los consumidores.
- Canal de comunicación: es un canal por el cual los consumidores y productores se envian/reciben mensajes o eventos.

Ejercicio 8

Describir la relación entre SLF4J (Simple Logging Facade for Java) y Fachada (4-6 líneas)

u otra

El SLF4J es una fachada que le permite al cliente o otra parte del sistema tewner un facil uso de los Logs, abstrayendo toda la complejidad de la logica de los logs. Que básicamente dependiendo del tipo de log que tenga que utilizar la parte del cliente el SLF4J mostrara mas o menos informacion en el Log.



Parte	Tema	Aspectos a evaluar
V	REST y UML	Identificar correctamente el concepto de "REST"

Ejercicio 9

9.1 Por qué la restricción *stateless* es fundamental en REST?

Porque la solicitudes que implementa REST hacia los servidores contienen toda la información necesaria para que el servidor pueda procesarla y elaborar una respuesta, en pocas palabra el servidor no guarda nunca la sesión del cliente que realiza la solicitud de un servicio. si, pero pregunta por qué es fundamental que sea así

9.2 Explicar el impacto que tiene sobre la escalabilidad y el diseño del servidor.



Ejercicio 10

¿Por qué la arquitectura del proyecto es REST? (4 líneas)
Básicamente se utilizo REST en el proyecto por los 3 principios que brinda:

- Principio de Ubicación de los recursos: quiere decir que los recursos que están en la Web tiene que ser fácilmente localizados por medio de una URI, contenido un path, un puerto, un esquema, entre otras cosas.
- Principio de Interfaz Uniforme:
  Basicamente como utiliza HTTP, lo que te permite que tengas un numero finitos de métodos para manipular los datos. (DELETE, GET, POST, PUT) todos métodos diferentes diferentes para modificar los datos.
- Principio de Representacion de los datos: básicamente por que hay diferentes formas de representarlos al mismo recurso por JSON o XML.

